

universidade de aveiro



deti

departamento de eletrónica,  
telecomunicações e informática

# Complements of Machine Learning

2025/2026



# Object Detection

---

# Today's Agenda

---

| Time          | Topic                                                                                   |
|---------------|-----------------------------------------------------------------------------------------|
| 13:00 - 14:15 | Theory: Object Detection & YOLO                                                         |
| 14:15 - 14:30 |  Break |
| 14:30 - 16:00 | Lab: Build an Object Detector with YOLOv8                                               |

# Learning Objectives

By the end of this session, you will:

1. **Understand** the difference between classification, localization, and detection
2. **Explain** how YOLO detects multiple objects in a single pass
3. **Evaluate** detectors using IoU, precision, recall, and mAP
4. **Build** a working object detector using YOLOv8

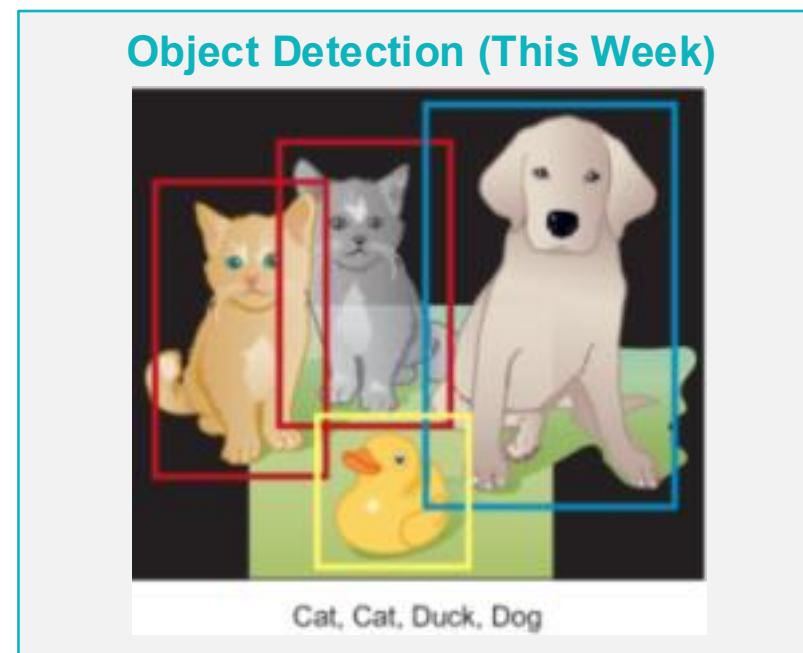
# 1. From Classification to Detection

What Changes When We Need to Find Objects?

# Classification vs Object Detection



One label per image



Multiple labels + bounding boxes

Adapted from: <https://livebook.manning.com/book/deep-learning-for-vision-systems/chapter-7>

# The Progression of Visual Understanding

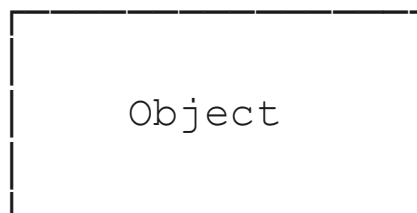
Each step adds more spatial understanding ↓

| Task                  | Output                  | Example                        |
|-----------------------|-------------------------|--------------------------------|
| Classification        | Single label            | "Cat"                          |
| Localization          | Label + 1 bounding box  | "Cat" at (x, y, w, h)          |
| Object Detection      | Multiple labels + boxes | "Cat" at (...), "Dog" at (...) |
| Segmentation (Week 4) | Pixel-level labels      | Every pixel classified         |

# What Is a Bounding Box?

A rectangle that tightly encloses an object.

`(x_min, y_min)`



Object

height (h)

`(x_max, y_max)`

| Format | Representation                            | Example                           |
|--------|-------------------------------------------|-----------------------------------|
| Corner | <code>(x_min, y_min, x_max, y_max)</code> | <code>(50, 30, 200, 180)</code>   |
| Center | <code>(x_center, y_center, w, h)</code>   | <code>(125, 105, 150, 150)</code> |

**YOLO uses center format, normalized to `[0, 1]`.**



# The Detection Challenge

---

## Why is detection harder than classification?

### 1. Variable number of outputs

Classification: always 1 label

Detection: 0, 1, 5, or 100 objects

### 2. Localization

Must predict WHERE objects are

Continuous coordinates, not discrete

### 3. Multiple scales

Large and small objects in same image

Person foreground vs car background

### 4. Overlapping objects

Objects can occlude each other

Handling partial visibility

# 2. Detection Approaches

Two-Stage vs One-Stage

# Two-Stage Detectors: R-CNN Family

## The Two-Stage Idea

Image → [Region Proposals] → [Classify Each Region]

Step 1: "Where MIGHT objects be?" → ~2000 candidate regions

Step 2: "What IS in each region?" → CNN classifies each proposal

| Model        | Year | Speed        | Key Innovation                    |
|--------------|------|--------------|-----------------------------------|
| R-CNN        | 2014 | ~47 s/image  | CNN on each proposal              |
| Fast R-CNN   | 2015 | ~2 s/image   | Share CNN features                |
| Faster R-CNN | 2015 | ~0.2 s/image | Learnable Region Proposal Network |

✅ High accuracy    ❌ Slow (two stages), complex pipeline

# One-Stage Detectors: The YOLO Philosophy

## *"You Only Look Once"*

Image → [Single Neural Network] → All Detections

One forward pass through the network



All bounding boxes + classes + confidences

**Key Insight: Treat detection as a single regression problem.**

| Detector     | FPS  | Use Case                 |
|--------------|------|--------------------------|
| Faster R-CNN | ~7   | Accuracy-critical        |
| SSD          | ~46  | Balanced                 |
| YOLOv3       | ~30  | Real-time                |
| YOLOv8       | ~80+ | Real-time, high accuracy |

# Why YOLO Matters

## Before YOLO (2015)

- Detection was slow
- Not practical for real-time
- Video processing nearly impossible

## After YOLO

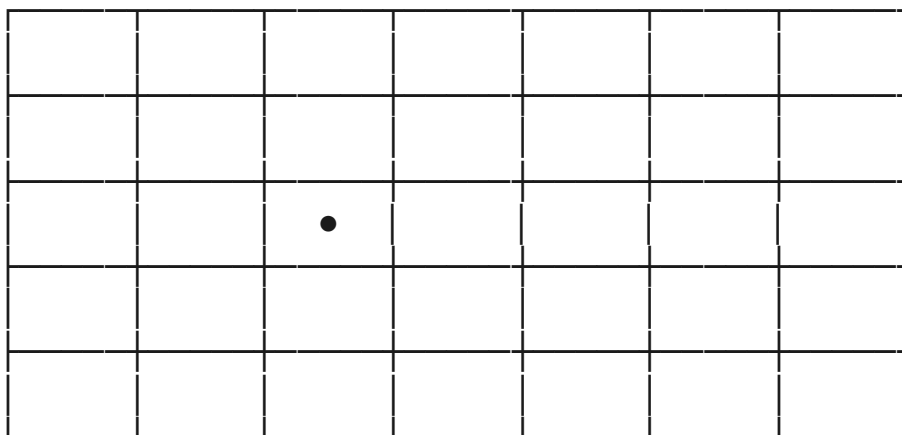
- Real-time detection on video streams
- Autonomous driving, surveillance, robotics
- Detection became accessible to everyone

***"YOLO made object detection fast enough to be useful."***

# 3. How YOLO Works

Grid-Based Detection

# Step 1: Divide the Image into a Grid



7 × 7 grid

## Key Rules:

- If an object's center falls in a grid cell, that cell is responsible for detecting it
- Each cell predicts B bounding boxes (YOLOv1: B=2)
- The grid divides the spatial reasoning across the image

# Step 2: Each Cell Predicts Bounding Boxes

## For each bounding box:

- x, y (center, relative to cell)
- w, h (size, relative to image)
- conf (P(Object) × IoU)

## For each cell:

- C class probabilities

**Total output per cell:  $B \times 5 + C$  values**

### YOLOv1 (PASCAL VOC)

Grid:  $7 \times 7$

B = 2 boxes/cell

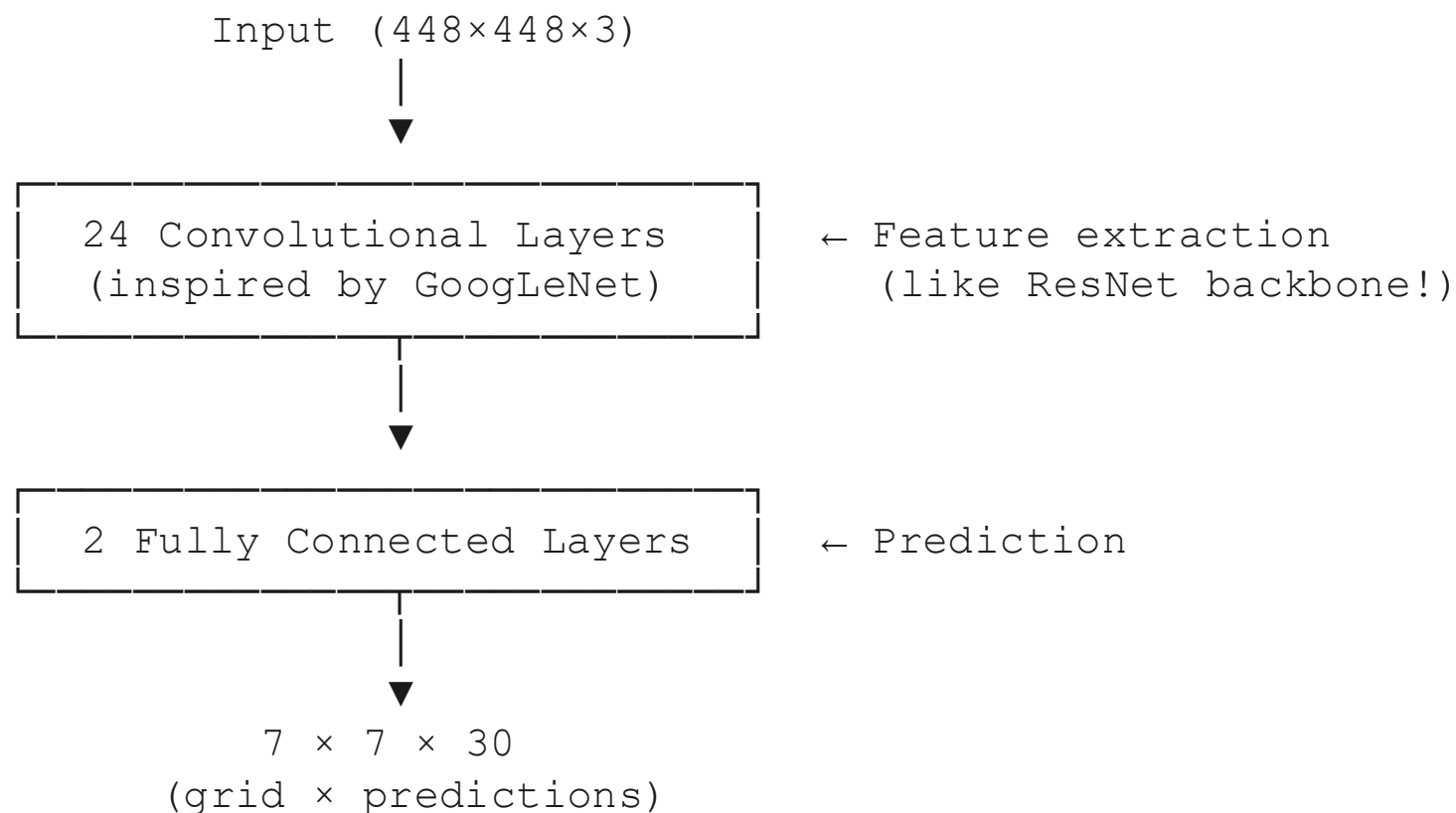
C = 20 classes

Per cell:  $2 \times 5 + 20 = 30$

Total:  $7 \times 7 \times 30 = 1470$  values



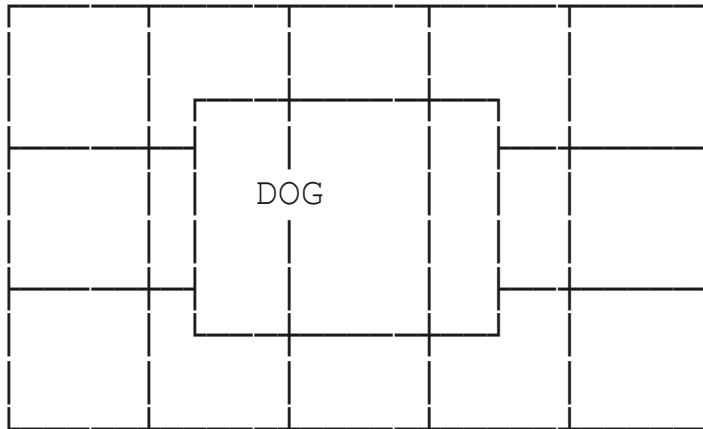
# Step 3: Network Architecture (YOLOv1)



**One single forward pass — hence "You Only Look Once"**

# Visual Example: YOLO in Action

Original Image with Grid Overlay:



**Cell (2,1) is responsible:**

Dog's center falls in this cell

```
Predicts:
(x, y, w, h, conf=0.95,
 class="dog")
```

# From YOLOv1 to YOLOv8: Key Improvements

---

| Version | Year | Key Innovation                                    |
|---------|------|---------------------------------------------------|
| YOLOv1  | 2016 | Single-pass detection, grid-based                 |
| YOLOv2  | 2017 | Batch normalization, anchor boxes, multi-scale    |
| YOLOv3  | 2018 | Feature Pyramid Network (FPN), 3 detection scales |
| YOLOv4  | 2020 | Bag of specials, CSPDarknet backbone              |
| YOLOv5  | 2020 | PyTorch-native, easy-to-use API                   |
| YOLOv8  | 2023 | Anchor-free, decoupled head, state-of-the-art     |

# YOLOv8 Architecture Overview

## BACKBONE (CSPDarknet + C2f)

Extracts features at multiple scales: P3 (small obj) | P4 (medium) | P5 (large)



## NECK (FPN + PAN)

Combines features from different scales (top-down + bottom-up paths)



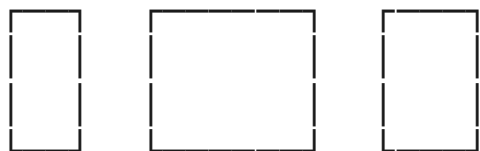
## HEAD (Decoupled, Anchor-Free)

Classification head + Regression head at each scale — No anchor boxes!

# Anchor Boxes vs Anchor-Free

## Anchor Boxes (YOLOv2-v5)

Pre-defined shapes:



tall      wide      medium

Predict offsets from anchors  
Sensitive to anchor design

## Anchor-Free (YOLOv8)

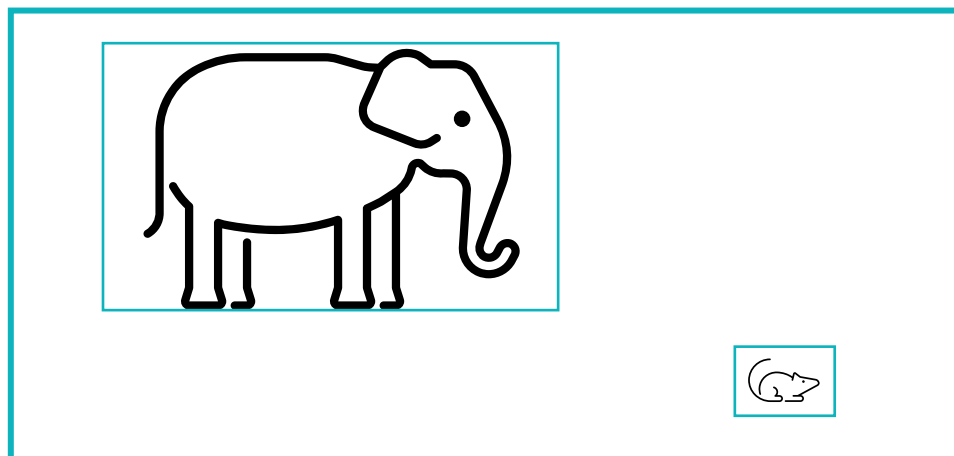
Predict directly:

Each point predicts:  
(left, top, right, bottom)  
distances to box edges

- ✓ Simpler, more flexible
- ✓ Better generalization

# Multi-Scale Detection

## Why is this important?



← Large object (coarse grid)

← Small object (fine grid)

| Scale       | Feature Map | Stride | Best For       |
|-------------|-------------|--------|----------------|
| P3 (Large)  | 80×80       | 8      | Small objects  |
| P4 (Medium) | 40×40       | 16     | Medium objects |
| P5 (Small)  | 20×20       | 32     | Large objects  |

# The YOLO Loss Function

YOLOv8 optimizes three losses simultaneously:

$$\text{Total Loss} = \lambda_{\text{box}} \times L_{\text{box}} + \lambda_{\text{cls}} \times L_{\text{cls}} + \lambda_{\text{dfl}} \times L_{\text{dfl}}$$

## 1. Box Loss (CloU)

How well boxes  
match ground truth  
Considers overlap, center  
distance, AND aspect ratio

## 2. Classification (BCE)

Binary Cross-Entropy per class  
Multi-label: object can trigger  
multiple classes

## 3. Distribution Focal (DFL)

Learns distribution over box  
boundaries for better  
localization precision

# Bounding Box Encoding (YOLO Format)

**YOLO normalizes all values to [0, 1]:**

Image (640×480):

Object at pixel coordinates:

x\_min=100, y\_min=50, x\_max=300, y\_max=200

YOLO format (class x\_center y\_center width height):

|   |         |         |         |         |
|---|---------|---------|---------|---------|
| 0 | 0.3125  | 0.2604  | 0.3125  | 0.3125  |
|   | ↑       | ↑       | ↑       | ↑       |
|   | 200/640 | 125/480 | 200/640 | 150/480 |

**Label file format (one line per object):**

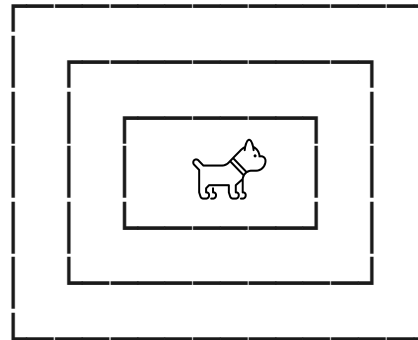
| class_id | x_center | y_center | width  | height |
|----------|----------|----------|--------|--------|
| 0        | 0.3125   | 0.2604   | 0.3125 | 0.3125 |
| 2        | 0.7500   | 0.6000   | 0.2000 | 0.3000 |



# 4. Non-Maximum Suppression

Removing Duplicate Detections

# The Problem: Duplicate Detection



`conf = 0.72`  
`conf = 0.89`  
`conf = 0.95`

**3 detections for 1 object — we only want the best one!**

## Solution: Non-Maximum Suppression (NMS)

1. Sort all detections by confidence (descending)
2. Pick the top detection → keep it
3. Remove all remaining detections overlapping above IoU threshold
4. Repeat until no repeat detections remain

# NMS: Worked Example

---

## Before NMS:

Box A: conf=0.95  
Box B: conf=0.89  
Box C: conf=0.72  
Box D: conf=0.60

## After NMS:

Box A: conf=0.95 ✓  
Box D: conf=0.60 ✓

## Process:

$\text{IoU}(A, B) = 0.82 \rightarrow$  suppress B (same object)  
 $\text{IoU}(A, C) = 0.75 \rightarrow$  suppress C (same object)  
 $\text{IoU}(A, D) = 0.10 \rightarrow$  keep D (different object!)

# NMS Parameters and Trade-offs

| Parameter            | Low Value               | High Value                  |
|----------------------|-------------------------|-----------------------------|
| Confidence threshold | More detections (noisy) | Fewer detections (may miss) |
| IoU threshold (NMS)  | Aggressive suppression  | More overlapping boxes kept |

## YOLOv8 Defaults:

Confidence = 0.25 → Keep boxes with confidence > 0.25

IoU threshold = 0.45 → Suppress boxes overlapping > 45%

## Tuning for your application:

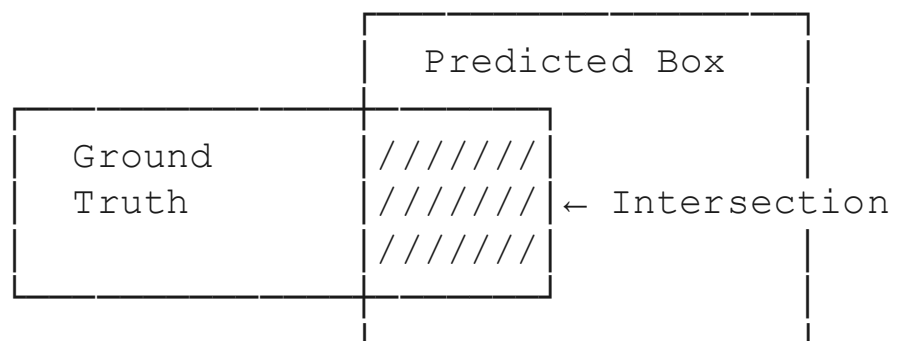
- Crowded scenes (people in a crowd): Higher IoU threshold → allow more overlap
- Sparse scenes (cars on highway): Lower IoU threshold → aggressive suppression
- Safety-critical (medical): Lower confidence → don't miss anything

# 5. Evaluation Metrics

Measuring Detection Quality

# Intersection over Union (IoU)

The fundamental metric for bounding box quality:



$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

| IoU  | Interpretation                      |
|------|-------------------------------------|
| 0.0  | No overlap                          |
| 0.5  | "Loose" match (PASCAL VOC standard) |
| 0.75 | "Strict" match                      |
| 1.0  | Perfect overlap                     |

# IoU: Calculating Step by Step

Ground Truth: (50, 50, 200, 200) → Area =  $150 \times 150 = 22,500$

Prediction: (80, 60, 220, 210) → Area =  $140 \times 150 = 21,000$

Intersection:

x\_left =  $\max(50, 80) = 80$

y\_top =  $\max(50, 60) = 60$

x\_right =  $\min(200, 220) = 200$

y\_bottom =  $\min(200, 210) = 200$

Intersection area =  $(200 - 80) \times (200 - 60) = 120 \times 140 = 16,800$

Union =  $22,500 + 21,000 - 16,800 = 26,700$

IoU =  $16,800 / 26,700 = 0.629$

**True Positive at IoU  $\geq 0.5$  threshold ( $0.629 > 0.5$ ) ✓**

# Precision and Recall for Detection

## Definitions:

True Positive (TP): Detection with  $\text{IoU} \geq \text{threshold}$  matching ground truth

False Positive (FP): Detection with no matching ground truth (or duplicate)

False Negative (FN): Ground truth with no matching detection

### Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

"Of all detections,  
how many are correct?"

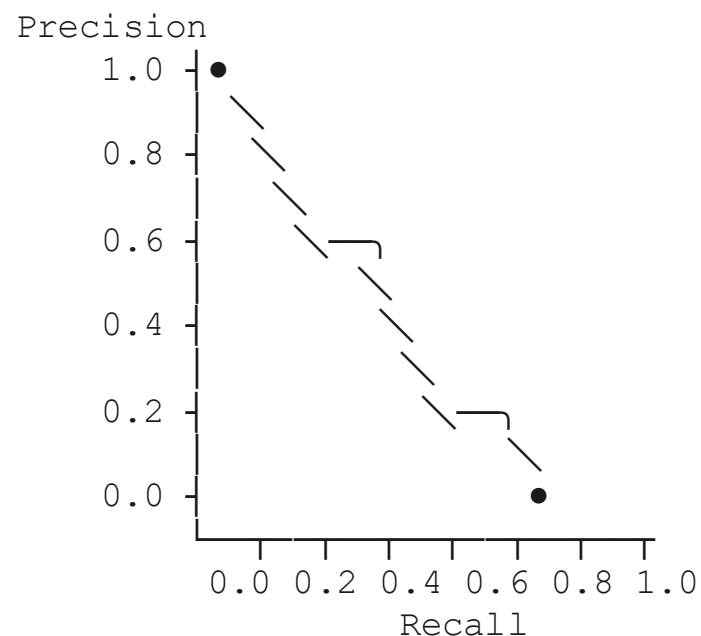
### Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

"Of all real objects,  
how many did we find?"



# Precision-Recall Curve & AP



## How to read it:

- High confidence threshold  
→ top-left (high precision, low recall)
- Low confidence threshold  
→ bottom-right (low precision, high recall)

AP = Area under this curve (for one class)

mAP = Average over all classes

# Mean Average Precision (mAP)

The standard metric for object detection benchmarks.

Step 1: For each class  $\rightarrow$  compute Precision-Recall curve

Step 2: AP = area under the curve (for each class)

Step 3:  $mAP = (1/C) \times \sum AP_c$

| Metric           | IoU Threshold                               | Used By             |
|------------------|---------------------------------------------|---------------------|
| mAP@0.5 (mAP50)  | 0.5                                         | PASCAL VOC          |
| mAP@0.75 (mAP75) | 0.75                                        | Stricter evaluation |
| mAP@[0.5:0.95]   | Avg over 0.5 $\rightarrow$ 0.95 (step 0.05) | COCO (primary)      |

**COCO mAP is the most rigorous — averages across 10 IoU thresholds**

# Detection Metrics Summary

---

| Metric         | What It Measures          | Good Value                      |
|----------------|---------------------------|---------------------------------|
| IoU            | Box quality               | > 0.5 (OK), > 0.75 (good)       |
| Precision      | Detection correctness     | > 0.8                           |
| Recall         | Object coverage           | > 0.8                           |
| mAP@0.5        | Overall detection quality | > 0.7 (good), > 0.9 (excellent) |
| mAP@[0.5:0.95] | Strict overall quality    | > 0.5 (good), > 0.7 (excellent) |
| FPS            | Speed                     | > 30 (real-time)                |

# 6. Practical Considerations

Real-World Object Detection

# Object Detection in Practice

## Common Pitfalls

### 1. Class Imbalance

Background  $\gg$  objects.  
YOLO handles internally with focal strategies.

### 2. Small Objects

Hardest to detect (few pixels).  
Solution: larger input resolution or P3 head.

### 3. Dense Scenes

Many overlapping objects.  
May need higher NMS IoU threshold.

### 4. Domain Gap

COCO model may fail on medical.  
Solution: fine-tune on your domain!

# Speed vs Accuracy: YOLOv8 Model Sizes

| Model            | Params | mAP@[0.5:0.95] | Speed (T4) |
|------------------|--------|----------------|------------|
| YOLOv8n (nano)   | 3.2M   | 37.3           | 1.21 ms    |
| YOLOv8s (small)  | 11.2M  | 44.9           | 1.40 ms    |
| YOLOv8m (medium) | 25.9M  | 50.2           | 2.26 ms    |
| YOLOv8l (large)  | 43.7M  | 52.9           | 2.99 ms    |
| YOLOv8x (xlarge) | 68.2M  | 53.9           | 3.73 ms    |

## How to choose:

Edge / mobile → YOLOv8n or v8s | Server real-time → v8m | Accuracy-critical → v8l or v8x

# YOLOv8 Isn't Just for Detection

| Task            | Description           | Output                        |
|-----------------|-----------------------|-------------------------------|
| Detection       | Find objects          | Bounding boxes + class labels |
| Segmentation    | Instance masks        | Per-object pixel masks        |
| Pose Estimation | Human keypoints       | Skeleton joints               |
| Classification  | Image-level labels    | Single class per image        |
| Tracking        | Video object tracking | IDs + boxes across frames     |

**We focus on detection today — segmentation is next week!**

# When to Use Object Detection

## Good use cases

- Autonomous driving
- Security / surveillance
- Retail analytics
- Medical imaging (lesion detection)
- Manufacturing (defect detection)
- Wildlife monitoring

## Classification is enough X

- "Is there a cat?" → Classification
- "What flower is this?" → Classification
- Single-object, centered images

**Rule of thumb: If you need to know WHERE objects are, use detection.**



# Summary: Key Concepts

---

| Concept           | Key Idea                                         |
|-------------------|--------------------------------------------------|
| Object Detection  | Classify + localize multiple objects             |
| Two-Stage (R-CNN) | Region proposals → classify (accurate but slow)  |
| YOLO (One-Stage)  | Grid-based single pass (fast and accurate)       |
| Anchor-Free       | Predict box edges directly (YOLOv8)              |
| Multi-Scale       | Detect at 3 feature map resolutions              |
| NMS               | Remove duplicate overlapping detections          |
| IoU               | Overlap quality metric ( $\geq 0.5$ = match)     |
| mAP               | Avg. Precision across classes and IoU thresholds |



15 minutes

---

# Hands-On Lab

Build an Object Detector with YOLOv8 (90 minutes)

---

# Summary

What We Covered Today

---

# Today's Key Takeaways

1. **Object detection** = classification + localization for multiple objects
  2. **YOLO philosophy**: single pass, grid-based, real-time speed
  3. **YOLOv8**: anchor-free, multi-scale, state-of-the-art
  4. **NMS** removes duplicate detections using IoU overlap
  5. **mAP** is the standard detection metric
  6. **Practical**: Ultralytics API makes detection accessible in a few lines
-

# Looking Ahead

---

## Next Week: Image Segmentation

- From bounding boxes to pixel-level masks
- U-Net architecture (encoder-decoder)
- Semantic, instance, and panoptic segmentation
- Loss functions: Dice loss, focal loss
- Lab: Medical image segmentation

## Preparation

- Reading: U-Net paper (Ronneberger et al., 2015)
- Review: Transposed convolutions (upsampling)

# Questions?

---

**Q: Can YOLO detect objects it wasn't trained on?**

A: No — fine-tune on new classes, or use zero-shot detectors (CLIP-based, covered later).

**Q: How many images for custom training?**

A: Minimum ~50/class for decent results. 200+/class is a good target.

**Q: YOLO vs Faster R-CNN — when to use each?**

A: YOLO for real-time; Faster R-CNN when accuracy > speed (medical, satellite).

**Q: Can YOLO work on non-image data?**

A: Core algorithm is for images/video, but spectrograms, radar plots, etc. can be treated as images.

# Thank You!

See you next week!

Lab submission due in 1 week

---